

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: «Коммивояжёр: динамическое программирование »

Студент гр. 4345

Андросов М.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить алгоритмы для решения задачи коммивояжёра. Реализовать алгоритм АЛШ-2.

Задание

Напишите программу, решающую задачу коммивояжера. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

- n - количество городов ($5 \leq n \leq 15$).
- Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j . Если $graph[i][j]=0$ (и $i \neq j$), это означает, что прямого пути между городами нет.

Выходные данные:

- Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.
- Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в начальном городе.
- Если такого пути не существует, вывести "no path".

Sample Input 1:

```
5
0 1 13 23 7
12 0 15 18 28
21 29 0 33 28
23 19 34 0 38
5 40 7 39 0
```

Sample Output 1:

```
78
0 4 2 3 1 0
```

Sample Input 2:

3
0 1 0
1 0 1
0 1 0

Sample Output 2:

no path

Вариант 8.

Точный метод: динамическое программирование (не МВиГ), итеративная реализация. Приближённый алгоритм: АЛШ-2.

Выполнение работы

Для решения задачи был разработан алгоритм Хелда-Карпа , а также выполнен приближенный алгоритм АЛШ-2.

Описание алгоритма Хелда-Карпа

Вместо хранения списка посещенных городов алгоритм использует двоичную маску , где i -ый бит равен 1 если город I посещен. Таблица состояний($dp[mask][curr]$) хранит минимальную стоимость пути, который начинается в стартовом городе, проходит через все города из $mask$, заканчивается в $curr$. Алгоритм перебирает все маски и для каждого $(mask, curr)$ пытается перейти в непосещённые города $next$. Если новый путь дешевле , обновляет dp и запоминает предыдущую вершину в $parent$. После обработки всех масок перебираются все возможные последние вершины из которых может быть выполнен возврат в стартовый город и вычисляется минимальная стоимость замкнутого маршрута. Если оптимальная стоимость равна бесконечности то гамильтонов цикл отсутствует в графе и программа выводит "no path". С помощью массива $parent$ алгоритм собирает полный маршрут.

Сложность по времени: $O(n^2 \cdot 2^n)$ (Всего 2^n масок и для каждой перебирается n городов, для каждого из них требуются до n следующих).

Пространственная сложность: $O(n \cdot 2^n)$.

Описание алгоритма АЛШ-2

Создается массив `visited[n]`. Стартовый город отмечается как посещенный. На каждом шаге алгоритм перебирает все города v . Для каждого подходящего v вычисляется оценка ($est = graph[current][v] + prim_mst(graph, buf)$), где `prim_mst` — это вес минимального остовного дерева, построенного на оставшемся множестве городов. Среди всех v выбирается тот, у которого est — минимально. После проверки n городов проверяется наличие дороги из последнего города в стартовый. Программа выводит последовательность городов (`path`) и итоговую стоимость (`total_cost`).

Временная сложность: $O(n^4)$ (выполняется основной цикл $n-1$ раз, затем перебираются все непосещенные вершины v — до n раз, для каждого подходящего v запускается алгоритм Прима — сложность $O(k^2)$, где k — кол-во вершин подграфа)

Пространственная — $O(n^2)$

Разработанный программный код см. в приложении А.

Тестирование

Таблица 1 — Результаты тестирования алгоритма Хелда-Карпа .

№	Входные данные	Выходные данные
1	4 0 10 15 20 10 0 35 25 15 35 0 30 20 25 30 0	80 0 2 3 1 0
2	4 0 10 0 25 10 0 15 0 0 15 0 20 25 0 20 0	70 0 3 2 1 0
3	5 0 10 0 0 0 10 0 20 0 0 0 20 0 30 0 0 0 30 0 40 0 0 0 0 0	No path

Таблица 2 — Результаты тестирования алгоритма АЛШ-2.

№	Входные данные	Выходные данные
1	4 0 79.86 19.16 78.19 79.86 0 45.14 10.9 19.16 45.14 0 15.14 78.19 10.9 15.14 0	Path: 0→2→3→1→0 Cost: 125.06
2	3 0 79.86 19.16 78.19 0 60.09 45.14 10.9 0	Path: 0→2→1→0 Cost: 108.25

Вывод

Были реализованы алгоритмы Хелда-Карпа и АЛШ-2 для эффективного решения задачи коммивояжера.